LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark DataFrame

Module 02 — Transformations & Column Operations

Expert Class Notes

Complex DataFrames | Narrow vs Wide | select() | col() lit() concat() | cast() date_format() | withColumn() | Rename | drop() | distinct() | dropDuplicates() | Null Handling

Real Dataset	Column Ops	Transformations	Data Quality
Nested types, nulls, dates	col, lit, concat, cast	select, withColumn, rename	distinct, dropDup, dropna

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years of industry experience, Sateesh is the founder of LevelUp Cloud Trainings — India's premier data engineering education platform with 5,000+ trained students.

DataFrame 02 expert notes cover the complete column transformation toolkit — from creating production-grade complex DataFrames with nested types to mastering select, withColumn, rename, deduplication, and null handling.

Core Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: Creating Complex DataFrames — Real-World Schema

In production data engineering, DataFrames are never just flat tables with simple columns. Real datasets have nested structs, arrays, boolean flags, date/timestamp fields, and null values. This module teaches you to handle all of these using the exact dataset from class.

> **Sateesh Says 💡**
>
> *Every real Indian tech company — Zomato, Flipkart, Paytm — stores user data with nested phone numbers, order history arrays, and nullable fields.*
>
> *If you can model and query this complex schema, you can handle ANY production dataset.*
>
> *The users dataset from class has EVERY complex type you'll encounter: Row (struct), List (array), boolean, date, datetime, and nulls. Perfect training ground!*

1.1 The Users Dataset — Complete Schema Analysis

This is the exact dataset taught in class. Study each field carefully — the field types are deliberately chosen to cover every real-world data type scenario.

Creating the complex users DataFrame — from class

```
import datetime
from pyspark.sql import Row, SparkSession
import pandas as pd

spark = SparkSession.builder.appName('UsersDataFrameExample').getOrCreate()

users = [
    {
        'id': 1,
        'first_name': 'Rahul',
        'last_name': 'Sharma',
        'email': 'rahul.sharma@example.com',
        # Row() creates a STRUCT type — nested object
        'phone_numbers': Row(mobile='+91 98765 43210',
                             home='+91 98765 43211'),
        'courses': [101, 102],    # Array<Integer>
        'is_customer': True,      # BooleanType
        'amount_paid': 1500.75,   # DoubleType
        'customer_from': datetime.date(2022, 5, 12),   # DateType
        'last_updated_ts': datetime.datetime(2022, 6, 10, 9, 30, 0) # TimestampType
    },
    # ... rows 2-3: normal data
    {  # Row 4 — NULLS everywhere — important for null handling
        'id': 4, 'first_name': 'Kavita', 'last_name': 'Iyer',
        'email': 'kavita.iyer@example.biz',
        'phone_numbers': Row(mobile=None, home=None),  # null struct
        'courses': [],              # EMPTY array
        'is_customer': False,       # boolean false
        'amount_paid': None,        # null numeric
```

```
            'customer_from': None,      # null date
            'last_updated_ts': datetime.datetime(2022, 11, 10, 14, 35, 30)
        },
        # Row 5: partial nulls in struct
        {  'id': 5, ...,
           'phone_numbers': Row(mobile='+91 91234 56789', home=None),
           'amount_paid': None, 'customer_from': None
        }
    ]

    # Convert via Pandas (bridges Python dicts to Spark)
    users_pd_df = pd.DataFrame(users)
    users_df = spark.createDataFrame(users_pd_df)
    users_df.show(truncate=False)
```

1.2 Schema Deep Dive — printSchema() Explained

printSchema output and what each type means

```
users_df.printSchema()

# root
#  |-- id:               long        (nullable = true)
#  |-- first_name:       string      (nullable = true)
#  |-- last_name:        string      (nullable = true)
#  |-- email:            string      (nullable = true)
#  |-- phone_numbers:    struct      (nullable = true)  <-- NESTED!
#  |    |-- mobile:      string      (nullable = true)
#  |    |-- home:        string      (nullable = true)
#  |-- courses:          array       (nullable = true)  <-- ARRAY!
#  |    |-- element:     long        (containsNull = true)
#  |-- is_customer:      boolean     (nullable = true)
#  |-- amount_paid:      double      (nullable = true)
#  |-- customer_from:    date        (nullable = true)
#  |-- last_updated_ts:  timestamp   (nullable = true)
```

Field	PySpark Type	Python Source	Notes
id	LongType	int	Python int -> Spark long (64-bit)
first_name	StringType	str	Standard string — UTF-8
phone_numbers	StructType	Row(a=v, b=v)	Nested struct — access with dot notation
courses	ArrayType(Long)	[101, 102]	Array — access with getItem(index)
is_customer	BooleanType	True/False	Boolean — filter with == True
amount_paid	DoubleType	1500.75	Double precision float — for money use Decimal
customer_from	DateType	datetime.date(...)	Date only (no time) — yyyy-MM-dd
last_updated_ts	TimestampType	datetime.datetime(...)	Full timestamp with time

1.3 Accessing Nested and Array Fields

Querying struct and array columns

```
# Access struct field with dot notation
users_df.select('phone_numbers.mobile').show()
users_df.select(col('phone_numbers.mobile').alias('mobile')).show()

# Access array element by index (0-based)
users_df.select('courses').show()           # shows full array
users_df.select(col('courses')[0]).show()   # first element only
users_df.select(col('courses').getItem(0).alias('course1')).show()

# Explode array — one row per element
from pyspark.sql.functions import explode
users_df.select('id', explode('courses').alias('course_id')).show()
# User with courses=[101,102] becomes 2 rows: (1,101) and (1,102)

# Filter by array size
from pyspark.sql.functions import size
users_df.filter(size(col('courses')) > 0).show()  # enrolled users
users_df.filter(size(col('courses')) == 0).show() # no courses
```

1.4 Interview Questions — DataFrame Creation

IQ-01 **What is the difference between DateType and TimestampType in PySpark?**

DateType stores only the date portion (year-month-day). TimestampType stores full date + time including seconds and microseconds. Use DateType for birthday, joining date, or any date-only field. Use TimestampType for event logs, transaction times, or anything needing time precision. In Python: datetime.date() creates DateType, datetime.datetime() creates TimestampType. For analytics, DateType is more storage efficient.

IQ-02 **What is a StructType (struct) column in PySpark? How do you access its fields?**

A StructType is a nested column containing multiple sub-fields — like an object within a row. Created from Row() objects or explicitly defined with StructType([StructField(...)]). Access sub-fields with dot notation: col('phone_numbers.mobile') or 'phone_numbers.mobile' as a string. In SQL: SELECT phone_numbers.mobile FROM users. StructType is common for storing address objects, contact info, or any hierarchically related fields.

IQ-03 **What is the difference between createDataFrame from a list of dicts vs list of tuples?**

List of dicts: column names come from dictionary keys — order doesn't matter, flexible but slower to parse. List of tuples: column names must be provided as second argument — faster but order-sensitive. List of Row() objects: most flexible, supports nested types.

Via Pandas: best for complex types (struct, array) and handles None/NaN correctly. Explicit schema (StructType) is fastest and most production-appropriate — avoids type inference scan.

Module 2: Narrow vs Wide Transformations — The Big Picture

Every PySpark operation is either a narrow or wide transformation. This classification determines whether a shuffle occurs, which directly impacts performance. Understanding this is mandatory for senior data engineering roles and architecture discussions.

> **Sateesh Says 💡**
>
> *Think of a narrow transformation like each Zomato delivery boy doing their own zone — no communication needed between zones. Fast, parallel, independent.*
>
> *Think of a wide transformation like the Zomato city manager reshuffling ALL delivery boys across ALL zones on peak day — everything has to stop, regroup, then restart.*
>
> *The shuffle is the regroup step. It is the most expensive operation in Spark. Minimize it by applying narrow transformations FIRST to reduce data volume.*

2.1 Narrow Transformations — No Shuffle, No Data Movement

Each output partition depends on ONLY ONE input partition. Data stays on the same Executor. No network transfer occurs. Fast, cheap, and can be pipelined together.

Transformation	Operation	Example
df.select	Pick specific columns	df.select('id','first_name')
df.filter / where	Filter rows by condition	df.filter(col('amount_paid') > 1000)
df.withColumn	Add or modify a column	df.withColumn('tax', col('amount_paid')*0.18)
df.withColumnRenamed	Rename a column	df.withColumnRenamed('id','user_id')
df.drop	Remove columns	df.drop('email','phone_numbers')
df.map (RDD)	Transform each element	rdd.map(lambda x: x.upper())
df.flatMap (RDD)	Transform and flatten	rdd.flatMap(lambda x: x.split())
df.limit	Take first N rows	df.limit(10)

2.2 Wide Transformations — Shuffle Required

Output partition depends on MULTIPLE input partitions. Data must be redistributed across the network (shuffle). Expensive — creates a stage boundary in the DAG. Always try to narrow the data BEFORE a wide transformation.

Transformation	Operation	Why Shuffle?
df.distinct	Remove duplicates	All identical rows must meet on same node
df.union / set ops	Combine DataFrames	Data from different partitions merged
df.join	Combine on key	Matching keys must land on same Executor
df.groupBy + agg	Group and aggregate	All same-key rows must be on same node
df.sort / orderBy	Sort rows globally	Global sort requires all data to be seen
df.repartition	Change partition count	Full data redistribution
df.crossJoin	Cartesian product	Every row meets every other row

2.3 The Golden Pipeline Rule

Always apply narrow transformations BEFORE wide ones

```
# BAD PATTERN: groupBy on full dataset with all 50 columns
df.groupBy('city').agg(F.sum('amount_paid'))  # shuffles ALL columns!

# GOOD PATTERN: select needed columns FIRST, then groupBy
df.select('city','amount_paid') \                  # narrow (fast)
  .filter(col('amount_paid').isNotNull()) \        # narrow (fast)
  .groupBy('city').agg(F.sum('amount_paid'))        # wide (shuffle)

# Why? After select+filter, the shuffled dataset is ~5% of original.
# The shuffle moves 95% less data. 20x faster!

# Catalyst Optimizer does SOME of this automatically (predicate
# pushdown, column pruning) — but being explicit is always faster.
```

2.4 Interview Questions — Narrow vs Wide

IQ-04 **What is the difference between narrow and wide transformations?**

Narrow: each output partition depends on ONLY ONE input partition — no data movement, no shuffle. Examples: filter, select, withColumn, map. Wide: each output partition depends on MULTIPLE input partitions — requires shuffle (data redistribution across the network). Examples: groupBy, join, distinct, orderBy. Wide transformations are more expensive because shuffling involves network I/O, disk spill, and serialization overhead.

IQ-05 **Is df.filter() a narrow or wide transformation? Explain why.**

df.filter() is a NARROW transformation. Each row is evaluated independently — does it meet the condition? Yes: keep. No: discard. The result for any output partition depends only on the corresponding input partition. No row from partition 0 can influence the content of partition 1's filter output. Therefore no shuffle is needed. filter() runs in parallel

on all partitions simultaneously — one of the cheapest and most performant operations.

IQ-06 Why is df.distinct() considered a wide transformation?

distinct() needs to find ALL duplicate rows across the ENTIRE dataset. A duplicate row could have its copies split across different partitions (e.g., duplicate ('Raj','Delhi') in partition 0 AND partition 3). To identify and remove the duplicate, both copies must be brought to the same node. This requires a shuffle. Internally it is implemented similarly to a groupBy on all columns followed by keeping one representative.

Module 3: select() — Every Pattern You Need

select() is the most-used DataFrame operation. It creates a new DataFrame with specified columns — applying the Catalyst Optimizer's column pruning to skip reading unnecessary columns from disk. Knowing all 8 patterns separates junior from senior engineers.

> **Sateesh Says** 💡
>
> *select() is the SELECT clause of SQL — but distributed across a petabyte-scale cluster.*
>
> *The MOST important thing about select(): it triggers column pruning in Catalyst — if you don't select a column, Spark NEVER reads it from Parquet/Delta files.*
>
> *At Flipkart: their orders Parquet has 200 columns. If you only select 5 — Spark reads 2.5% of the data. 97.5% I/O saved instantly!*

3.1 All 8 Ways to Use select() — From Class

Pattern 1: Wildcard — select all columns

```
# Select ALL columns
users_df.select('*').show()
users_df.select(users_df.columns).show()  # equivalent

# With alias — create a named reference to the DataFrame
users_df.alias('u').select('u.*').show()
# 'u.*' means 'all columns from alias u' — same as '*' but explicit
```

Pattern 2: String column names

```
# Simple, clean — most readable
users_df.select('id', 'first_name', 'last_name').show()

# Using a Python list (useful when columns are dynamic)
cols = ['id', 'first_name', 'last_name']
users_df.select(cols).show()        # list directly
users_df.select(*cols).show()       # unpack with *
```

Pattern 3: col() function — from class

```
from pyspark.sql.functions import col

users_df.select(col('id'), col('first_name'), col('last_name')).show()

# Mixed: col(), df['col'], string — all work together
users_df.select(users_df['id'], col('first_name'), 'last_name').show()
```

Pattern 4: With alias, concat, lit — from class

```
from pyspark.sql.functions import col, concat, lit

# concat first_name + ', ' + last_name -> full_name
users_df.select(
    'id', 'first_name', 'last_name',
    concat(users_df['first_name'], lit(', '), col('last_name'))
        .alias('full_name')
).show()

# Another variation from class:
users_df.select(
    col('id'),
    col('first_name'),
    col('last_name'),
).show()
```

Output — with full_name

```
+---+----------+---------+--------------+
| id|first_name|last_name|     full_name|
+---+----------+---------+--------------+
|  1|     Rahul|   Sharma| Rahul, Sharma|
|  2|     Anita|    Verma|  Anita, Verma|
|  3|    Sanjay|    Patel| Sanjay, Patel|
+---+----------+---------+--------------+
```

Pattern 5: selectExpr — SQL expressions as strings (from class)

```
# selectExpr accepts any SQL expression string
users_df.selectExpr(
    'id',
    'first_name',
    'last_name'
).show()

# Power of selectExpr: full SQL inside Python code
users_df.selectExpr(
    'id',
    'upper(first_name) as first_name_upper',
    'amount_paid * 0.18 as gst_amount',
    'CASE WHEN is_customer THEN "Yes" ELSE "No" END as customer_flag',
    'year(customer_from) as joining_year'
).show()
```

Pattern 6: Spark SQL via createOrReplaceTempView — from class

```
# Register DataFrame as a SQL temporary view
users_df.createOrReplaceTempView('users')

# Now query with pure SQL
spark.sql('''
    SELECT id, first_name, last_name
    FROM users
''').show()

# Full SQL power — SAME performance as DataFrame API!
spark.sql('''
    SELECT id,
           CONCAT(first_name, ', ', last_name) AS full_name,
           DATE_FORMAT(customer_from, 'dd-MMM-yyyy') AS joined_on,
           CASE WHEN is_customer THEN 'Active' ELSE 'Inactive' END
                AS status
```

```
    FROM users
    WHERE amount_paid IS NOT NULL
    ORDER BY amount_paid DESC
''').show()
```

Pattern 7: Exclude certain columns (no built-in 'select except')

```
# PySpark has no SELECT * EXCEPT — use these workarounds

# Method 1: List comprehension — exclude specified columns
exclude = ['phone_numbers', 'courses', 'last_updated_ts']
users_df.select([c for c in users_df.columns if c not in exclude]).show()

# Method 2: drop() — cleaner for few columns to remove
users_df.drop('phone_numbers', 'courses').show()

# Method 3: Delta Lake EXCEPT (if using Delta Lake tables)
spark.sql('SELECT * EXCEPT(phone_numbers, courses) FROM users').show()
```

Pattern 8: Dynamic column selection using df.columns

```
# Select all STRING columns only
string_cols = [f.name for f in users_df.schema.fields
                if str(f.dataType) == 'StringType()']
users_df.select(string_cols).show()

# Select all non-null columns (runtime check)
from pyspark.sql.functions import count, when
non_null_cols = [c for c in users_df.columns
    if users_df.filter(col(c).isNull()).count() == 0]
users_df.select(non_null_cols).show()
```

3.2 Interview Questions — select()

IQ-07 **What is the difference between select() and selectExpr()?**

select() takes Column objects, column name strings, or expressions built using the col() API. selectExpr() takes SQL expression strings directly — allowing CASE WHEN, date functions, arithmetic, string functions all in SQL syntax inside Python code. Both produce identical execution plans via Catalyst — same performance. Use select() for programmatic column operations; use selectExpr() when the expression is complex and easier to read as SQL.

IQ-08 **Why does select() improve performance compared to reading all columns?**

PySpark integrates with columnar formats (Parquet, ORC, Delta). When you select() specific columns, Catalyst Optimizer applies column pruning — it reads ONLY those specific columns from disk, skipping all others. A Parquet file storing 100 columns but you select 5: Spark reads approximately 5% of the file. For a 1 TB dataset this saves ~950 GB of disk I/O. This optimization is one of the main reasons Parquet is preferred over CSV in production.

IQ-09 What is the difference between select() and df['column']?

df['column'] returns a Column object (a reference to a column) — it does NOT trigger computation. It is used to build column expressions. select() is a transformation that creates a new DataFrame containing the specified columns — it is lazy and builds the execution plan. Think of df['column'] as picking up a tool, and select() as using that tool to build something.

Module 4: col(), lit(), concat() — Column Functions

These three functions from pyspark.sql.functions are the building blocks of every column expression in PySpark. You cannot write production PySpark without mastering these.

4.1 col() — Reference a Column Programmatically

Sateesh Says 💡

col() is like pointing at a column by name — 'I want THAT column'.

Unlike df['colname'], col() is not tied to any specific DataFrame — it's more flexible in complex expressions.

In 99% of cases, col('name'), df['name'], and 'name' (string) produce identical results.

But col() is REQUIRED inside F.when(), F.sort_array(), complex nested expressions — strings won't work there!

col() — All important patterns

```
from pyspark.sql.functions import col

# Basic column reference
users_df.select(col('id'), col('first_name')).show()

# 3 equivalent ways to reference a column — from class
users_df.select(users_df['id'], col('first_name'), 'last_name').show()
# All three are equivalent in this context

# col() in filter conditions
users_df.filter(col('amount_paid') > 1000).show()
users_df.filter(col('is_customer') == True).show()
users_df.filter(col('first_name').startswith('R')).show()

# col() in withColumn
users_df.withColumn('tax', col('amount_paid') * 0.18).show()

# col() with alias
users_df.select(col('id').alias('user_id')).show()

# col() for sorting
users_df.orderBy(col('amount_paid').desc()).show()
users_df.orderBy(col('last_name').asc(), col('id').desc()).show()

# Accessing nested struct fields using col() with dot notation
users_df.select(col('phone_numbers.mobile')).show()
users_df.select(col('phone_numbers').getField('mobile')).show()
```

4.2 lit() — Create a Constant (Literal) Column

lit() creates a column where every row has the same constant value. It is used to add fixed strings, numbers, or flags to a DataFrame, or inside expressions where you need a constant value.

lit() — From class and extended examples

```
from pyspark.sql.functions import lit

# From class: add a constant 'Country' column
data = [('Alice', 25), ('Bob', 30), ('Catherine', 45)]
df = spark.createDataFrame(data, ['Name', 'Age'])

df_with_country = df.withColumn('Country', lit('USA'))
df_with_country.show()
# +--------+---+-------+
# |    Name|Age|Country|
# +--------+---+-------+
# |   Alice| 25|    USA|
# |     Bob| 30|    USA|
# |Catherine| 45|    USA|

# lit() with various types
df.withColumn('flag',       lit(True)).show()    # boolean literal
df.withColumn('tax_rate',   lit(0.18)).show()    # float literal
df.withColumn('category',   lit('Premium')).show() # string literal
df.withColumn('version',    lit(1)).show()       # integer literal
df.withColumn('null_col',   lit(None)).show()    # null literal

# lit() inside concat — from class
users_df.select(
    concat(col('first_name'), lit(' '), col('last_name'))
    .alias('full_name')
).show()

# lit() inside when() for conditional constants
from pyspark.sql.functions import when
df.withColumn('status',
    when(col('Age') >= 18, lit('Adult')).otherwise(lit('Minor'))
).show()
```

4.3 concat() — Join String Columns Together

concat() merges two or more string columns (and literal strings) into one combined string. It is one of the most used functions in ETL for creating full names, composite keys, formatted messages, and report strings.

concat() — All patterns from class and production

```
from pyspark.sql.functions import col, concat, lit, concat_ws

# From class: first_name + '|' + last_name
users_df.select(
    col('id'),
    concat(users_df['first_name'], lit('|'), col('last_name')
           .alias('full_name'))
```

```
).show()

# Better version from class — alias OUTSIDE concat
users_df.select(
    'id', 'first_name', 'last_name',
    concat(users_df['first_name'], lit(', '), col('last_name'))
        .alias('full_name')
).show()

# concat_ws() — BETTER: concat With Separator
# concat_ws(separator, col1, col2, ...)
# BONUS: concat_ws IGNORES nulls! concat() with nulls = null result
users_df.select(
    concat_ws(' ', col('first_name'), col('last_name')).alias('name'),
    concat_ws(' | ', col('first_name'), col('email')).alias('label')
).show()

# concat() NULL behaviour — critical to know!
# If ANY argument is null, concat() returns null
# concat_ws() skips null arguments instead
null_df = spark.createDataFrame([('Ravi', None)], ['f', 'l'])
null_df.select(concat(col('f'), lit(' '), col('l'))).show()
# OUTPUT: null  <- because col('l') is null!
null_df.select(concat_ws(' ', col('f'), col('l'))).show()
# OUTPUT: Ravi  <- concat_ws skips the null!
```

Full Name Output

```
+---+----------+---------+--------------+
| id|first_name|last_name|     full_name|
+---+----------+---------+--------------+
|  1|     Rahul|   Sharma| Rahul, Sharma|
|  2|     Anita|    Verma|  Anita, Verma|
|  3|    Sanjay|    Patel| Sanjay, Patel|
|  4|    Kavita|    Iyer |  Kavita, Iyer|
|  5|      Ravi|    Nair |    Ravi, Nair|
+---+----------+---------+--------------+
```

4.4 Interview Questions — col, lit, concat

IQ-10 **What is the difference between col(), df['col'], and a plain string in select()?**

All three reference columns but have different scopes: String ('name'): simple, fast, but cannot be used in complex expressions or ambiguous multi-DataFrame joins. df['name']: tied to a specific DataFrame — useful when disambiguating join columns. col('name'): unambiguous column reference, works in any context including cross-DataFrame expressions, F.when(), complex nested calls. In single-DataFrame operations they are equivalent; use col() for consistency.

IQ-11 **What is lit() and when do you need it?**

lit() wraps a Python literal (string, int, float, bool, None) into a Spark Column object. You need it whenever you want to use a constant value in a column expression — concat(), when(), arithmetic with a constant. Without lit(), Spark cannot distinguish between a Python string 'India' (a column name reference) and the literal text 'India'. Example: concat(col('city'), lit(' - India')) — without lit() Spark would look for a column named ' - India'.

IQ-12 What is the difference between concat() and concat_ws()?

concat(c1, c2, ...) joins strings with NO separator. If ANY argument is null, the entire result is null. concat_ws(sep, c1, c2, ...) joins strings with a specified separator AND ignores null values — it simply skips nulls. Use concat_ws() in production whenever nullable columns are involved. concat(first_name, lit(' '), last_name) breaks when last_name is null; concat_ws(' ', first_name, last_name) safely returns just the first_name.

Module 5: cast() and date_format() — Type Conversion

Type conversion is one of the most critical operations in data engineering. Source data arrives in wrong types constantly — dates as strings, booleans as integers, amounts as strings. cast() and date_format() are your tools to fix this.

5.1 cast() — Convert Column Data Types

Sateesh Says 💡

In real ETL at HDFC Bank, transaction amounts come as strings from legacy systems: '50000', '75000.50'.

Before any calculation (sum, average, comparison), you MUST cast them to numeric type first.

cast() is the type conversion tool in PySpark — equivalent to CAST(col AS type) in SQL.

Wrong types = wrong results. Always verify schema with printSchema() and fix types immediately after loading data.

cast() — From class and all patterns

```
from pyspark.sql.functions import col
from pyspark.sql.types import IntegerType, DoubleType, StringType

# From class: explicit cast to int
data = [('Alice', 25), ('Bob', 30), ('Catherine', 45)]
df = spark.createDataFrame(data, ['Name', 'Age'])  # Age is string!

# Check schema — Age might be string
df.printSchema()   # |-- Age: long

# Cast using string type name (most common — concise)
df2 = df.select(col('Name'), col('Age').cast('int'))
df2.printSchema()  # |-- Age: integer

# Cast using Type class (explicit)
df2 = df.select(col('Name'), col('Age').cast(IntegerType()))

# Production ETL cast patterns
df.withColumn('salary',    col('salary_str').cast('double'))
df.withColumn('age',       col('age_str').cast('int'))
df.withColumn('is_active', col('is_active_int').cast('boolean'))
df.withColumn('join_date', col('join_date_str').cast('date'))
df.withColumn('txn_ts',    col('timestamp_str').cast('timestamp'))

# IMPORTANT: Failed cast returns null (no error!)
bad = spark.createDataFrame([('abc',), ('123',)], ['val'])
bad.select(col('val').cast('int')).show()
# 'abc' cast to int = NULL (not an error)
# '123' cast to int = 123
```

5.2 date_format() — Format Dates as Strings

date_format() converts a DateType or TimestampType column to a formatted string. This is essential for generating reports, creating partition keys, building composite IDs with dates, and displaying dates in regional formats.

date_format() — From class with all format patterns

```
from pyspark.sql.functions import date_format, col

# From class: format customer_from date as yyyyMMdd
users_df.select(
    col('id'),
    date_format('customer_from', 'yyyyMMdd')
).show()
# Output: 20220512, 20220723, 20220901, null, null

# From class: format AND cast to int (for partition key)
users_df.select(
    col('id'),
    date_format('customer_from', 'yyyyMMdd').cast('int')
        .alias('customer_from')
).show()
# Output integer: 20220512, 20220723, 20220901, null, null

# Verify the type:
users_df.select(
    col('id'),
    date_format('customer_from', 'yyyyMMdd').cast('int')
        .alias('customer_from')
).printSchema()
# |-- customer_from: integer (nullable = true)

# All common date format patterns:
users_df.select(
    date_format('customer_from', 'yyyy-MM-dd'),    # 2022-05-12
    date_format('customer_from', 'dd/MM/yyyy'),    # 12/05/2022
    date_format('customer_from', 'dd-MMM-yyyy'),   # 12-May-2022
    date_format('customer_from', 'MMMM dd, yyyy'), # May 12, 2022
    date_format('customer_from', 'E, dd MMM yyyy'),# Wed, 12 May 2022
    date_format('customer_from', 'yyyyMM'),        # 202205 (month)
    date_format('customer_from', 'yyyyMMdd'),      # 20220512 (int)
    date_format('last_updated_ts', 'HH:mm:ss'),    # time only
).show()
```

date_format output

```
+---+----------------------------------+
| id|date_format(customer_from, yyyyMM)|
+---+----------------------------------+
|  1|                          20220512|
|  2|                          20220723|
|  3|                          20220901|
|  4|                              null|
|  5|                              null|
+---+----------------------------------+
```

5.3 Other Important Column Type Functions

asc(), desc(), contains() — from class comments

```
# asc(), desc() — used in sort/orderBy
users_df.sort(col('amount_paid').desc()).show()         # high->low
users_df.orderBy(col('last_name').asc()).show()         # A->Z
users_df.orderBy(col('amount_paid').desc_nulls_last()).show() # nulls at end

# contains() — used in filter
users_df.filter(col('email').contains('example.com')).show()
users_df.filter(col('email').contains('@gmail')).show()

# like(), startswith(), endswith()
users_df.filter(col('first_name').startswith('R')).show()
users_df.filter(col('email').endswith('.in')).show()
users_df.filter(col('last_name').like('%ar%')).show()  # SQL LIKE

# between() — range filter (inclusive)
users_df.filter(col('amount_paid').between(500, 1500)).show()
users_df.filter(col('customer_from').between('2022-01-01','2022-06-30')).show()

# isin() — filter by list of values
users_df.filter(col('id').isin([1, 3, 5])).show()
users_df.filter(~col('id').isin([2, 4])).show()   # NOT IN
```

5.4 Interview Questions — cast and date_format

IQ-13 **What happens when cast() fails for a row (e.g., casting 'abc' to Integer)?**

PySpark cast() NEVER throws an error for invalid conversions. When it cannot convert a value, it silently returns NULL for that row. This is called a silent cast failure. Example: spark.createDataFrame([('abc',)],['v']).select(col('v').cast('int')).show() outputs null. This is a major data quality risk — always check for unexpected nulls after casting with: df.filter(col('cast_col').isNull()).count() and compare with before-cast null count.

IQ-14 **What is the difference between to_date() and date_format()?**

to_date() converts a STRING to a DateType column: to_date(col('date_str'), 'dd/MM/yyyy') -> DateType. date_format() converts a DateType/TimestampType to a STRING in a specified format: date_format(col('date'), 'yyyyMMdd') -> StringType. They are inverses: string -> date (to_date), date -> string (date_format). In ETL: use to_date() when loading, date_format() when exporting or creating partition paths like 'year=2022/month=05/day=12'.

Module 6: withColumn(), Rename & drop() — Column Management

This module covers the complete toolkit for adding, modifying, renaming, and removing columns. These are the backbone of every ETL transformation pipeline. Understanding when to use each method is a key interview differentiator.

> **Sateesh Says 💡**
>
> *In every real ETL at TCS or Wipro, you receive raw data and must reshape it: add tax columns, rename confusing field names, remove sensitive PII columns before publishing to data lake.*
>
> *withColumn = add or modify one column. withColumnRenamed = rename one column. toDF = rename all columns. drop = remove columns.*
>
> *The MOST COMMON senior interview question: 'What's the difference between withColumn and select with alias?' Know the answer cold!*

6.1 withColumn() — Add or Modify a Column

withColumn() takes a column name and a column expression. If the name matches an existing column, it REPLACES that column. If it's new, a new column is ADDED to the right side of the DataFrame.

withColumn() — All patterns from class

```
from pyspark.sql.functions import col, lit, upper, trim

# Pattern 1: Add a completely new constant column (from class)
df_with_country = df.withColumn('Country', lit('USA'))
# Adds 'Country' column with 'USA' for every row

# Pattern 2: Add derived column from arithmetic (from class)
df_with_column_replaced = df_with_column.withColumn(
    'Age', df_with_column['Age'] + 1   # increment Age by 1
)
# NOTE: 'Age' already exists -> withColumn REPLACES it!

# Pattern 3: Row-level transformations (from class comment)
df_transformed = df.withColumn('Age', df['Age'] * 2)
# Doubles every Age value — existing column replaced

# Pattern 4: Conditional (CASE WHEN)
from pyspark.sql.functions import when
users_df.withColumn('customer_tier',
    when(col('amount_paid') >= 1200, 'Gold')
   .when(col('amount_paid') >= 700,  'Silver')
   .when(col('amount_paid').isNotNull(), 'Bronze')
   .otherwise('Non-Customer')
).show()

# Pattern 5: String transformations
users_df.withColumn('first_name', upper(col('first_name')))
```

```
users_df.withColumn('email',       trim(col('email')))
users_df.withColumn('email_lower', col('email').cast('string'))

# Pattern 6: Type casting
users_df.withColumn('amount_paid', col('amount_paid').cast('int'))
# amount_paid: 1500.75 -> 1500 (truncated, not rounded!)

# Pattern 7: Date extraction
from pyspark.sql.functions import year, month, dayofmonth
users_df.withColumn('join_year',  year(col('customer_from')))
users_df.withColumn('join_month', month(col('customer_from')))
users_df.withColumn('join_day',   dayofmonth(col('customer_from')))
```

Common Mistake — Watch Out!

withColumn creates a new DataFrame each time — chaining many withColumn calls on large DataFrames creates a deep lineage. For 10+ column additions, use select() with all expressions instead for better Catalyst optimization.

withColumn('col', expr) where 'col' already exists SILENTLY replaces it. No warning. Check column names carefully!

Never do: df = df.withColumn('x', ...) in a loop 100 times. This causes StackOverflowError for large DataFrames. Use select() or reduce() instead.

6.2 Column Renaming — 4 Methods

PySpark gives you 4 different ways to rename columns. Knowing WHEN to use each is critical — wrong choice can hurt performance or cause maintainability issues.

Method 1: alias() in select — from class

```
# Best for: creating a new DataFrame with renamed columns
# From class:
df_renamed = df.select(df['Name'].alias('FullName'), df['Age'])
df_renamed.show()
# +--------+---+
# |FullName|Age|
# |   Alice| 25|
```

Method 2: withColumn() — rename via replace

```
# Use when you want to keep ALL other columns
# This is NOT efficient for JUST renaming — use withColumnRenamed
df.withColumn('FullName', col('Name')).drop('Name').show()
# 1. Adds 'FullName' = copy of 'Name'
# 2. Drops original 'Name'
# BETTER: use withColumnRenamed for simple renames
```

Method 3: withColumnRenamed() — from class (BEST for single rename)

```
# Most explicit — rename ONE column at a time
# From class:
df_renamed = df_with_column_replaced.withColumnRenamed('FullName','Name')
df_renamed = df_with_column_replaced.withColumnRenamed('Country','State')
```

```
# Chain for multiple renames
df.withColumnRenamed('Name',     'employee_name')  \
  .withColumnRenamed('Age',      'employee_age')   \
  .withColumnRenamed('Country', 'employee_country') \
  .show()
```

Method 4: toDF() — rename ALL columns at once (from class)

```
# Renames ALL columns in one shot using positional mapping
# From class:
df3 = df_with_renamed_column.toDF('Name', 'years', 'country')
df3.show()

# Rename using dict — production pattern
rename_map = {'Name': 'full_name', 'Age': 'age_years', 'Country': 'nation'}
df_renamed = df.toDF(*[rename_map.get(c, c) for c in df.columns])

# Or using select with comprehension
df_renamed = df.select(
    [col(c).alias(rename_map.get(c, c)) for c in df.columns]
)
```

6.3 drop() — Remove Columns

drop() — From class and all patterns

```
# From class — drop single column
df4 = df3.drop('country')
df4.show()

# From class — drop multiple columns
df5 = df3.drop('country', 'years')
df5.show()

# Drop using list
pii_cols = ['email', 'phone_numbers', 'amount_paid']
clean_df = users_df.drop(*pii_cols)

# Safe drop — only if column exists
if 'temp_col' in df.columns:
    df = df.drop('temp_col')

# drop is equivalent to select(all-except-dropped)
# Both are narrow transformations — no shuffle!
```

6.4 Interview Questions — withColumn, Rename, drop

IQ-15 **What is the difference between alias() in select and withColumnRenamed()?**

alias() renames a column expression WITHIN a select() call — you choose exactly which columns to include and their new names. The result DataFrame has only the

selected columns. withColumnRenamed() renames one column in the EXISTING DataFrame and returns a new DataFrame with ALL original columns (one renamed). Use alias() when you're already writing a select(). Use withColumnRenamed() when you want all columns but need to rename one.

IQ-16 If you call withColumn() with the same name as an existing column, what happens?

withColumn() REPLACES the existing column with the new expression silently. No error, no warning. Example: df.withColumn('Age', col('Age')*2) replaces the Age column with doubled values. This is how you apply in-place transformations. It is different from SELECT — you don't need to explicitly list all other columns. The position of the replaced column stays in the same place as the original.

IQ-17 When would you use toDF() instead of withColumnRenamed()?

Use toDF() when renaming MANY or ALL columns at once. toDF('col1','col2','col3') maps new names positionally to all existing columns — much cleaner than chaining 10 withColumnRenamed() calls. Use withColumnRenamed() for renaming just one or a few specific columns by name. toDF() requires you to provide names for ALL columns in order — if you miss one or get the order wrong, you'll silently mismatch column names.

Module 7: distinct() and dropDuplicates() — Deduplication

Duplicate data is one of the most common data quality problems in production. Understanding how PySpark identifies and removes duplicates — and the difference between distinct() and dropDuplicates() — is essential for building reliable data pipelines.

> **Sateesh Says 💡**
>
> *Imagine HDFC Bank's transaction log. Due to a system retry, the same transaction gets recorded 3 times. If you compute 'total spend per customer' without deduplication, the amount is 3x too high!*
>
> *In the class dataset: users with id=2, id=3, and id=4 all have exact duplicate rows. And user id=2 also has a row with DIFFERENT amount_paid (900 vs 1050) — same person, different data.*
>
> *distinct() removes EXACT duplicates (all fields must match). dropDuplicates(['id']) keeps one row per id regardless of other field values.*
>
> *Which one to use depends on your business question: 'are these IDENTICAL rows?' vs 'is this the SAME entity?'*

7.1 The Class Dataset — Studying the Duplicates

Understanding the duplicate data from class

```
# The users_df has 8 rows with duplicates:
users_df.count()   # 8 total rows

# Row 3 and Row 4 (id=3, Orelie Penney) are IDENTICAL
# Row 5 and Row 6 (id=4, Ashby Maddocks) are IDENTICAL
# Row 2 and Row 8 (id=2, Nikolaus) are SIMILAR but NOT identical:
#   Row 2: amount_paid=900.0, last_updated_ts=2021-02-18 03:33:00
#   Row 8: amount_paid=1050.0, last_updated_ts=2021-02-25 03:33:00

# So after distinct():
users_df.distinct().count()  # 6 rows (removed 2 exact duplicates)

# After dropDuplicates(['id']):
users_df.dropDuplicates(['id']).show()  # 5 rows (one per id)
# Note: for id=2, it keeps FIRST occurrence (amount=900, not 1050)
```

7.2 distinct() vs dropDuplicates() — Complete Reference

distinct() — remove exact duplicates

```
# distinct() removes rows where ALL columns are identical
users_df.distinct().count()    # 6 (8 - 2 exact duplicates)
users_df.distinct().show()
```

```
# distinct on SPECIFIC columns (find unique cities)
users_df.select('first_name').distinct().show()   # unique names

# Count distinct values per column
from pyspark.sql.functions import countDistinct
users_df.select(
    countDistinct('id').alias('unique_ids'),
    countDistinct('is_customer').alias('unique_status')
).show()
```

dropDuplicates() — from class with subset

```
# dropDuplicates() with NO arguments = same as distinct()
users_df.dropDuplicates().show()   # same as distinct()

# FROM CLASS: dropDuplicates with subset — key difference!
users_df.dropDuplicates(['id']).show()
# Keeps ONE row per unique 'id', regardless of other column values
# Result: 5 rows (one per id: 1,2,3,4,5)
# For id=2 (two rows, different amounts): FIRST row is kept

# Multiple columns subset
users_df.dropDuplicates(['id', 'is_customer']).show()
# Unique combination of id + is_customer

# Production pattern: keep LATEST record per id
# (dropDuplicates keeps first — order matters!)
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number
w = Window.partitionBy('id').orderBy(col('last_updated_ts').desc())
users_df.withColumn('rn', row_number().over(w)) \
        .filter(col('rn') == 1) \
        .drop('rn') \
        .show()
# This GUARANTEES keeping the LATEST row per id
```

distinct()	dropDuplicates(subset)
Compares ALL columns	Compares ONLY specified subset columns
No subset option	dropDuplicates(['id','city']) supported
Both exact copies removed	All rows with same subset key reduced to 1
'Are ALL fields identical?'	'Is this the same entity by key?'
Triggers shuffle (wide)	Triggers shuffle (wide)
Same as dropDuplicates() with no args	More powerful — subset control
Use for dedup of exact copies	Use for 'one row per key' deduplication

7.3 Interview Questions — distinct and dropDuplicates

IQ-18 What is the difference between distinct() and dropDuplicates()?

distinct() removes rows where ALL columns are identical — equivalent to SQL SELECT DISTINCT *. dropDuplicates() with no arguments does the same. dropDuplicates(['col1','col2']) removes rows where the specified SUBSET of columns is duplicate, keeping one representative row per unique key combination. Use distinct() for exact full-row deduplication. Use dropDuplicates(subset) for entity-level deduplication where you want one row per customer_id, product_id, etc.

IQ-19 How do you keep only the LATEST version of each record using PySpark?

Use Window functions: Window.partitionBy('id').orderBy(col('timestamp_col').desc()). Add row_number().over(window) as a column, then filter for row_number == 1. This guarantees you keep the latest record per id. dropDuplicates() alone cannot guarantee which row is kept — it takes the 'first' encountered row which depends on partition order. For SCD (Slowly Changing Dimension) patterns and upsert logic, the window row_number approach is the standard.

Module 8: Null Handling — dropna(), fillna(), na.fill()

NULL values are present in virtually every real-world dataset. Handling them correctly is a fundamental data engineering skill. This module covers the complete null-handling toolkit using the exact examples from class.

Sateesh Says 💡

Look at the class dataset: Kavita and Ravi have amount_paid=null, customer_from=null — they are not customers.

If you accidentally include their NULL amount_paid in a SUM — no problem, Spark ignores nulls in aggregations.

But if you JOIN on a null key — that row is SILENTLY EXCLUDED from inner joins. This causes mysterious data loss in production!

The class whiteboard example: ravi/sita/null rows show the 3 scenarios: all-null row, partial-null row, named null value ('Nul').

8.1 Understanding Nulls — The Class Example

The whiteboard null example from class

```
# Sateesh's whiteboard example during class:
#
# name        age    city
# ravi        25     mumbai     <- complete row
# sita        null   delhi      <- partial null (age missing)
# NULL        30     null       <- partial null (name + city null)
# Nul         null   null       <- 'Nul' is a STRING, not a SQL NULL!

# Creating this DataFrame:
from pyspark.sql.functions import col
data = [
    ('ravi', 25, 'mumbai'),
    ('sita', None, 'delhi'),
    (None, 30, None),
    ('Nul', None, None)   # 'Nul' is text, not null!
]
df = spark.createDataFrame(data, ['name', 'age', 'city'])
df.show()
```

Class whiteboard DataFrame

```
+----+----+------+
|name| age|  city|
+----+----+------+
|ravi|  25|mumbai|
|sita|null| delhi|
|null|  30|  null|
| Nul|null|  null|
+----+----+------+
Note: Row 4 'Nul' is a STRING value, NOT a SQL NULL!
```

8.2 dropna() — Remove Rows with Nulls

dropna() and na.drop() — All patterns from class

```
# na.drop() and dropna() are IDENTICAL — aliases of each other

# FROM CLASS — various null-drop patterns:

# 1. Drop rows where ANY column is null (strictest)
users_df.na.drop().show()         # drops rows with ANY null
users_df.dropna().show()          # same — alias
users_df.na.drop(how='any').show()# explicit 'any'
df.na.drop(how='any')             # from class notebook

# 2. Drop rows where ALL columns are null
users_df.na.drop(how='all').show()  # only rows that are entirely null
df.na.drop(how='all')               # from class notebook
df.dropna(how='all')                # from class notebook

# 3. Drop based on SPECIFIC columns (most useful!)
users_df.dropna(subset=['amount_paid']).show()
# Only drops rows where amount_paid is null
# Result: removes Kavita (id=4) and Ravi (id=5)

users_df.dropna(subset=['amount_paid','customer_from']).show()
# Drop rows where EITHER amount_paid OR customer_from is null

# 4. thresh parameter — keep rows with >= N non-null values
users_df.dropna(thresh=3).show()   # keep only rows with >= 3 non-nulls

# Count rows before and after:
print('Before:', users_df.count())             # 5
print('After any:', users_df.dropna().count()) # 3 (removes id=4,5)
```

8.3 fillna() / na.fill() — Replace Nulls with Values

fillna() — All patterns

```
# Fill ALL null values with a single value (type-safe)
users_df.fillna(0).show()           # all numeric nulls -> 0
users_df.fillna('Unknown').show()   # all string nulls -> 'Unknown'
users_df.fillna(False).show()       # all boolean nulls -> False

# Fill SPECIFIC columns with SPECIFIC values (most useful!)
users_df.fillna({
    'amount_paid':   0.0,
    'is_customer':   False,
    'first_name':    'Unknown',
    'customer_from': '2099-12-31'  # sentinel date for 'no date'
}).show()

# na.fill() is identical to fillna()
users_df.na.fill(0).show()
users_df.na.fill({'amount_paid': 0.0, 'first_name': 'Unknown'}).show()
```

```
# Fill with COMPUTED value (e.g., mean)
mean_amount = users_df.select(F.avg('amount_paid')).collect()[0][0]
users_df.fillna({'amount_paid': mean_amount}).show()

# coalesce() — fill null with first non-null from multiple options
users_df.withColumn('safe_amount',
    F.coalesce(col('amount_paid'), col('default_amount'), F.lit(0))
).show()
```

8.4 Null-Safe Operations — Advanced Patterns

Advanced null handling patterns

```
# Detect nulls
users_df.filter(col('amount_paid').isNull()).show()   # null rows
users_df.filter(col('amount_paid').isNotNull()).show()# non-null rows

# Count nulls per column (production profiling pattern)
null_counts = users_df.select([
    F.count(F.when(col(c).isNull(), c)).alias(c)
    for c in users_df.columns
])
null_counts.show()

# Null percentage per column
total = users_df.count()
null_pct = users_df.select([
    (F.count(F.when(col(c).isNull(), c)) / total * 100)
    .alias(f'{c}_null_%')
    for c in users_df.columns
])
null_pct.show()

# Null-safe equality (== returns null when comparing null==null)
df1.join(df2, df1.city.eqNullSafe(df2.city))  # null==null is TRUE

# na.replace() — replace specific NON-NULL values
users_df.na.replace(['example.com'], ['test.com'], subset=['email']).show()
```

8.5 Interview Questions — Null Handling

IQ-20 **What is the difference between na.drop(how='any') and na.drop(how='all')?**

how='any' drops a row if ANY column has a null value — even one null is enough to exclude the row. This is the strictest approach and often removes too many rows in sparse datasets. how='all' drops a row only if ALL columns are null — an entirely empty row. In the class dataset with name/age/city: how='any' drops sita (null age), null name row, Nul row = 2 rows dropped. how='all' keeps all rows since no row has ALL three columns null.

IQ-21 How does NULL behave in PySpark aggregations like sum() and avg()?

PySpark aggregation functions automatically IGNORE NULL values. sum('amount_paid') only sums non-null values. avg('amount_paid') computes mean of non-null values only (denominator is count of non-null rows, not total rows). count('amount_paid') counts non-null values. count('*') counts ALL rows including null-field rows. This is ANSI SQL behavior. Exception: collect_list() includes nulls, while collect_set() excludes them.

IQ-22 How would you find the percentage of null values in each column of a large DataFrame?

Profile pattern: df.select([(count(when(col(c).isNull(), c)) / df.count() * 100).alias(f'{c}_null_pct') for c in df.columns]).show(). This uses list comprehension to build one null% expression per column. For large DataFrames, cache first and compute count once. In Databricks, use dbutils.data.summarize(df) which shows null%, distinct count, and basic statistics for all columns in one shot.

Module 9: Master Reference — All Operations

9.1 Complete DataFrame 02 Operations Table

Operation	Category	Shuffle?	Key Syntax
select()	Narrow	No	df.select('c1',col('c2'),expr.alias('c3'))
selectExpr()	Narrow	No	df.selectExpr('c1','c2*2 as c2x2')
col()	Column ref	-	col('name') — unbound column reference
lit()	Column const	-	lit('India') — constant value column
concat()	String func	No	concat(col('f'),lit(' '),col('l'))
concat_ws()	String func	No	concat_ws(' ',col('f'),col('l')) — null-safe
cast()	Type convert	No	col('age').cast('int') or cast(IntegerType())
date_format()	Date func	No	date_format('date_col','yyyyMMdd')
withColumn()	Narrow	No	df.withColumn('new',expr) — add/replace col
withColumnRenamed()	Narrow	No	df.withColumnRenamed('old','new')
toDF()	Narrow	No	df.toDF('col1','col2','col3') — rename all
alias()	Column rename	-	col('name').alias('emp_name') in select
drop()	Narrow	No	df.drop('col1','col2')
distinct()	Wide	Yes	df.distinct() — removes all exact dups
dropDuplicates()	Wide	Yes	df.dropDuplicates(['id']) — by subset
dropna() / na.drop()	Narrow	No	df.dropna(how='any',subset=['col'])
fillna() / na.fill()	Narrow	No	df.fillna({'col':value,'col2':val2})
na.replace()	Narrow	No	df.na.replace(['old'],['new'],subset=['c'])

Sateesh Says 💡

You have now mastered the complete DataFrame 02 toolkit — every operation from Sateesh's class session.

Practice Challenge 1: Load the users dataset, add a 'full_name' column, format dates as 'dd-MMM-yyyy', add a 'customer_tier' column using when(), then remove PII columns (email, phone_numbers).

Practice Challenge 2: Load the duplicate users dataset, profile null counts per column, keep only the latest record per id using window functions, then save as Parquet.

Practice Challenge 3: Write a query using createOrReplaceTempView to find the top 3 customers by amount_paid with full_name formatted as 'First Last' and joining year.

Interview Tip: When asked about any column operation, always mention whether it is narrow or wide, whether Catalyst optimizes it, and give a real-world example.

All the best! — Sateesh Pabbathi, MCT | www.levelupedu.net